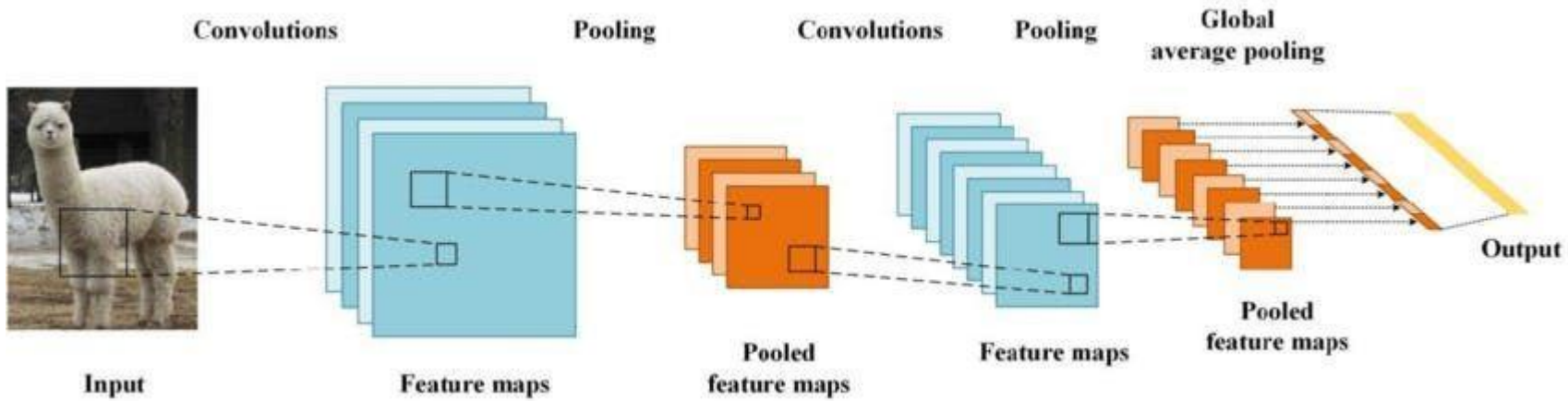
Artificial Intelligence and Machine Learning

6CS012 - Week 05 –Sunita Parajuli

Introduction
Convolution Neural Network



Tasks in Computer Vision

Is this a dog?

Image Classification

What is there in image and where?

Object Detection

Which pixels belong to which object?

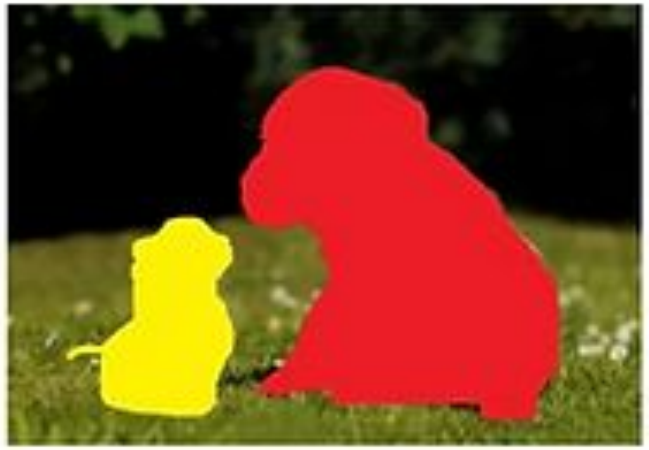
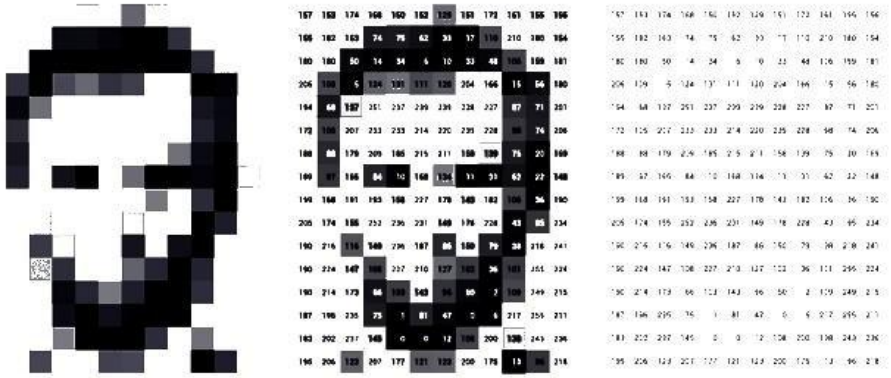
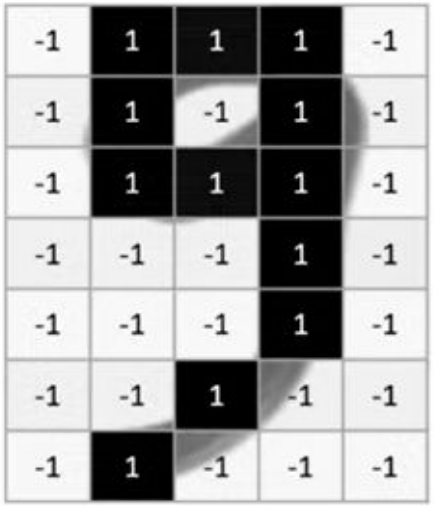


Image Segmentation

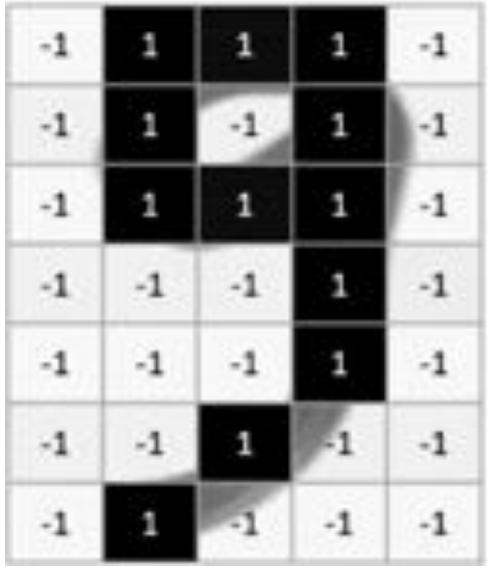
How do computers see images?

Images are just numbers



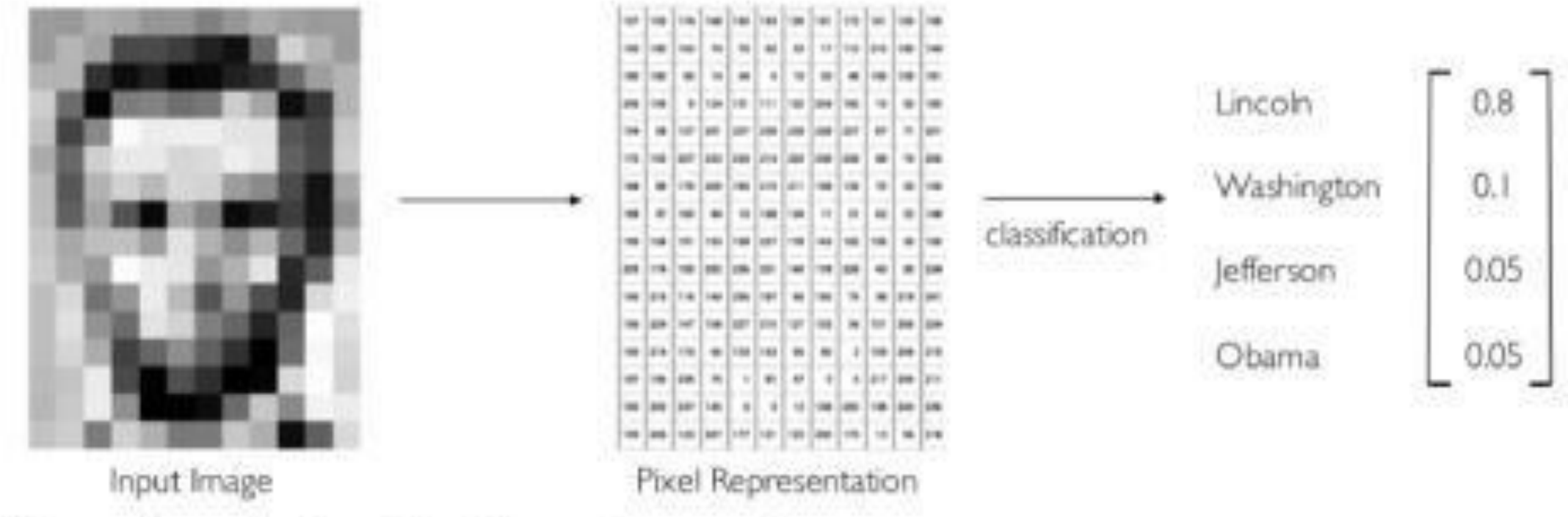


Deep Learning task with images



$P(0|x_i)$
$P(1|x_i)$
$P(2|x_i)$
$P(3|x_i)$
$P(4|x_i)$
$P(5|x_i)$
$P(6|x_i)$
$P(7|x_i)$
$P(8|x_i)$
$P(9|x_i)$

Deep Learning Tasks with Images



We need features to Identify the images

Lets Identify the features

- Nose
- Eyes
- Hair

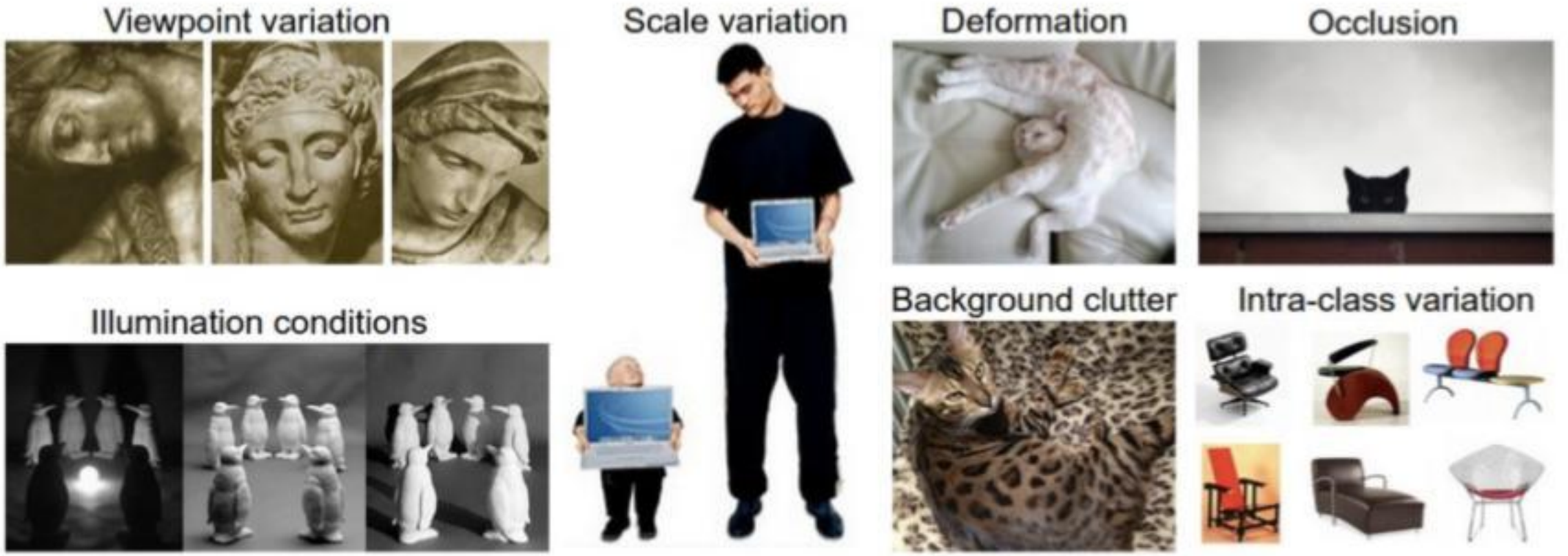
- Wheels
- License Plate
- Lights

- Doors
- Windows
- Roof

Manual Feature Extraction

- We need Domain Expertise
- Time-consuming
- Subjectivity
- Limited Scalability

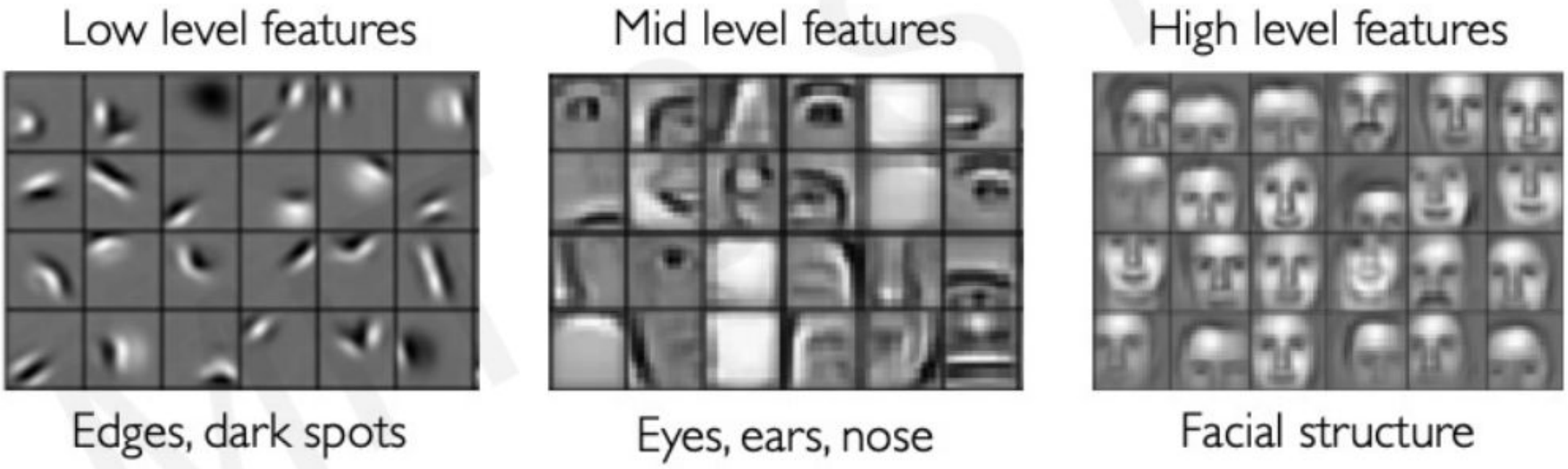
Manual Feature Extraction



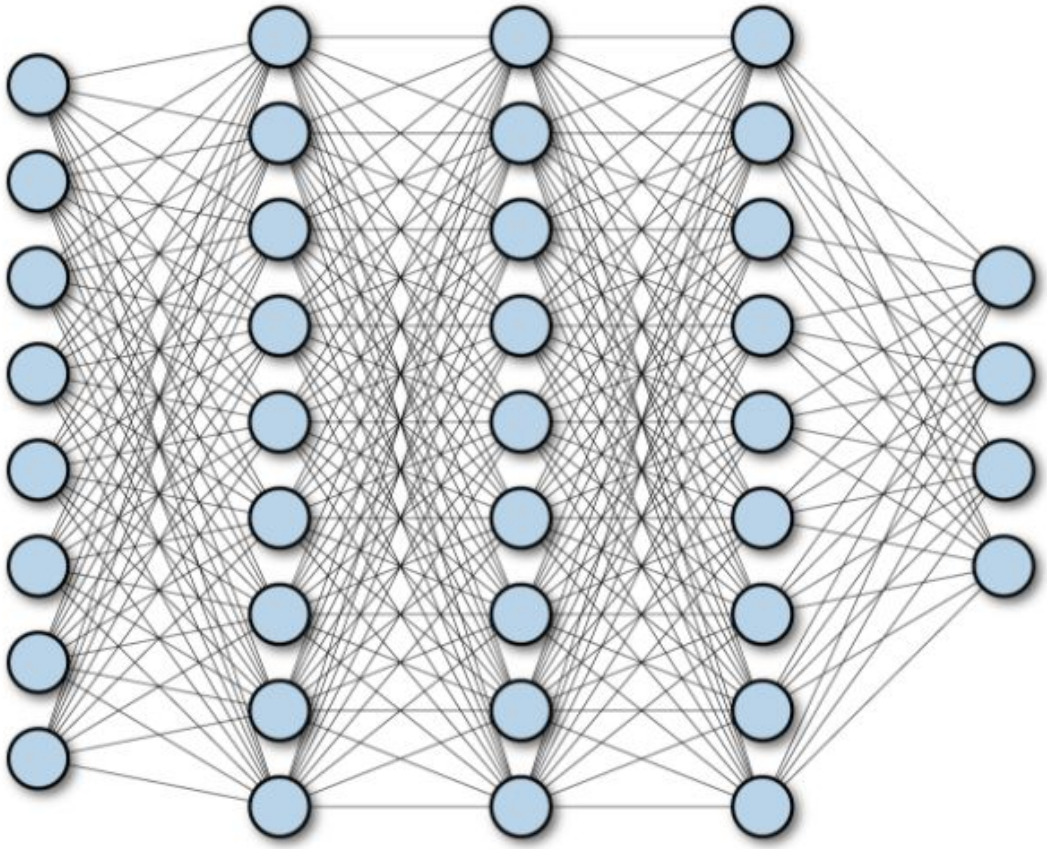
How do we learn the images without manually extracting features?

Learning Feature Representations

- Can we learn the hierarchical features directly from the data instead of hand engineering?



Fully Connected Neural Network



- **Automatic feature extraction**
- **Learns representations directly from data**
- **Reduces human bias**
- **More generalizable across domains**

Disadvantage of FCNN for image classification

Still Too Much Computation!!!!

Image size = 1920 x 1080 X 3

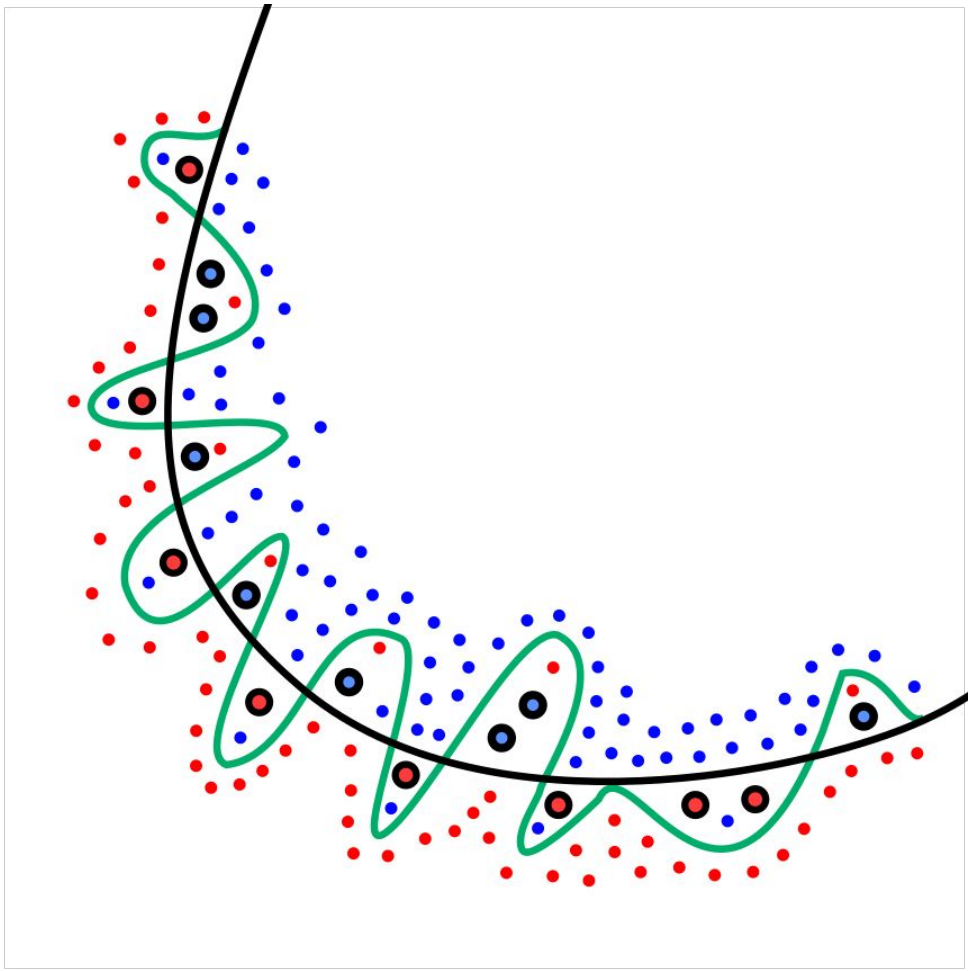
First layer neurons = 1920 x 1080 X 3 ~ 6 million

Hidden layer neurons = Let's say you keep it ~ 4 million

Weights between input and hidden layer = 6 mil * 4 mil
= 24 million

Why too much parameters is the problem?

- Computational Cost
- Model Overfitting



Why too much parameters is the problem?

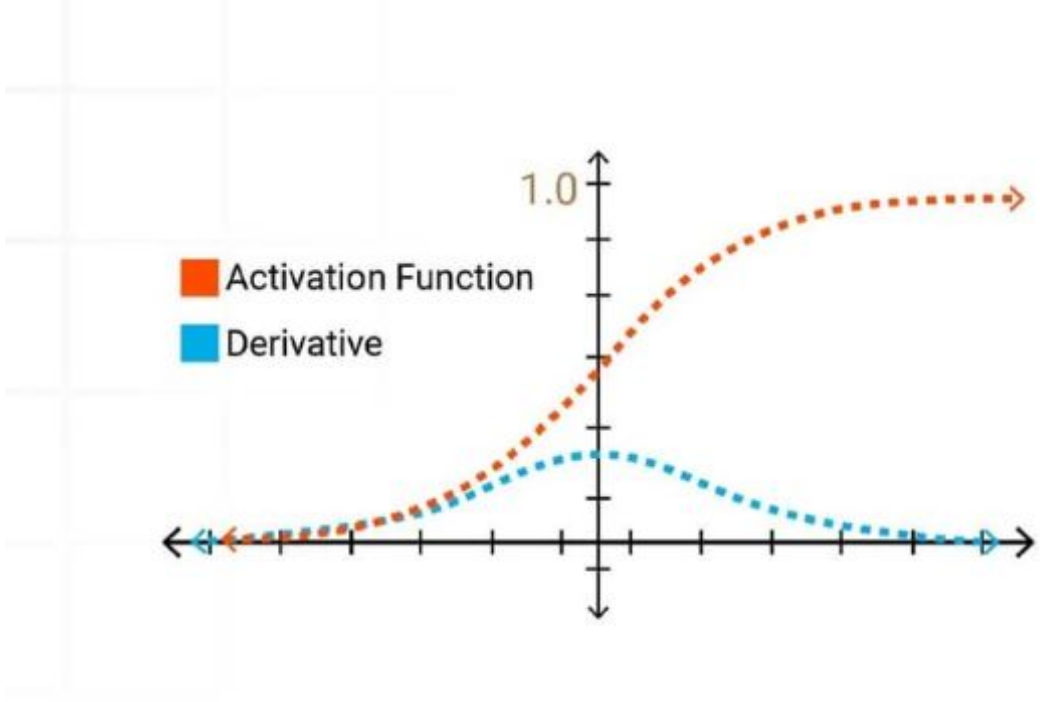
- Vanishing Gradient Problem (Example case :Sigmoid activation)

The sigmoid activation function is defined as: $\sigma(x) = \frac{1}{1+e^{-x}}$

Its derivative is given by: $\sigma'(x) = \sigma(x)(1 - \sigma(x))$

When $x \to \infty$, $\sigma(x) \to 1$, making $\sigma'(x) \approx 0$.

When $x \to -\infty$, $\sigma(x) \to 0$, making $\sigma'(x) \approx 0$.



Why too much parameters is the problem?

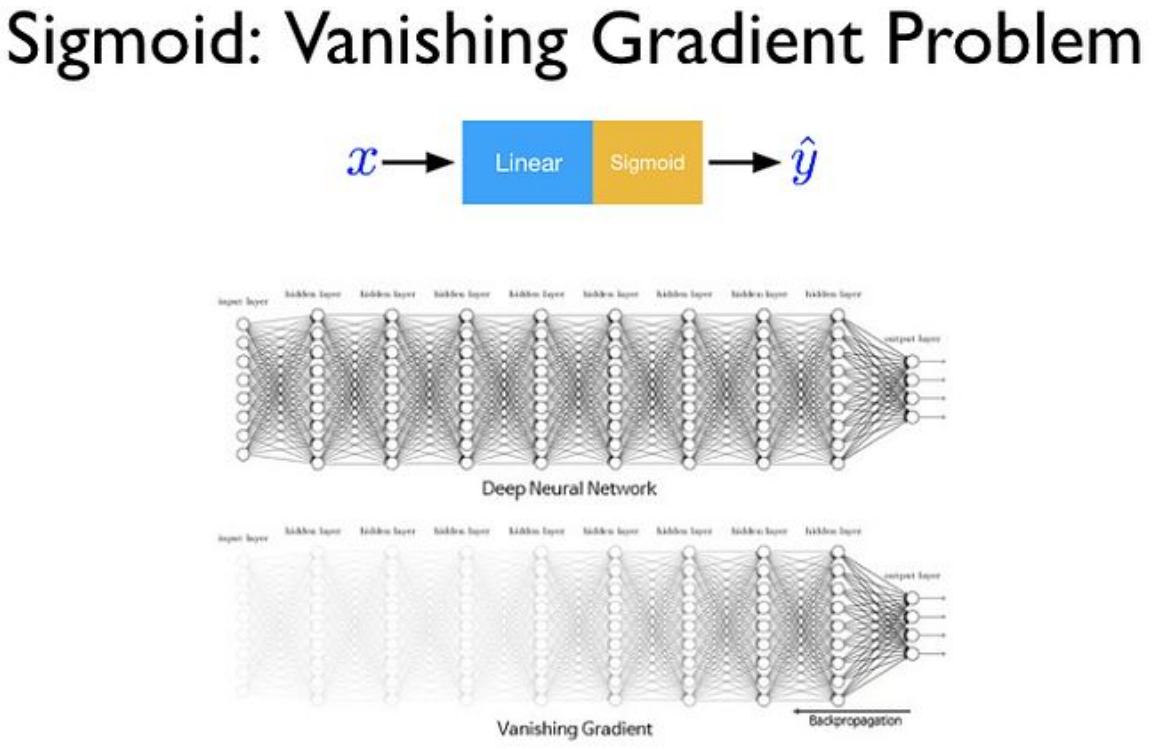
- Vanishing Gradient Problem (Example case :Sigmoid activation)

•Sigmoid function limits outputs to [0,1] leading to saturation at extreme values.

•In saturated regions, gradients approach zero, reducing weight updates.

•This issue is minor in shallow networks but worsens in deep networks due to gradient multiplication.

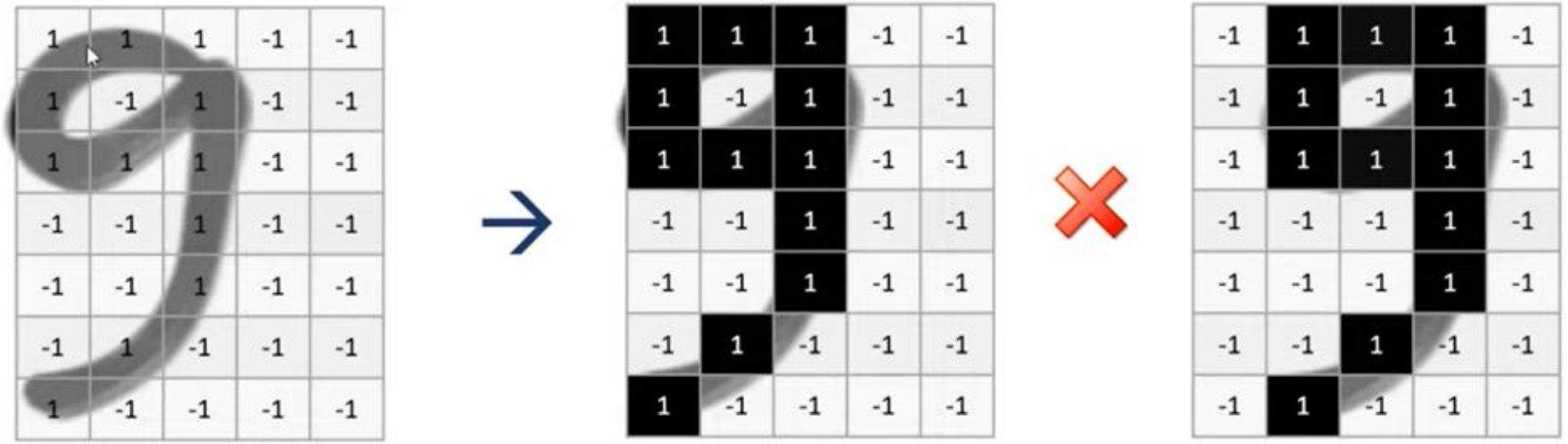
•As layers increase, gradients decay significantly, slowing training for early layers.



Disadvantage of FCNN for image classification

No Spatial Information

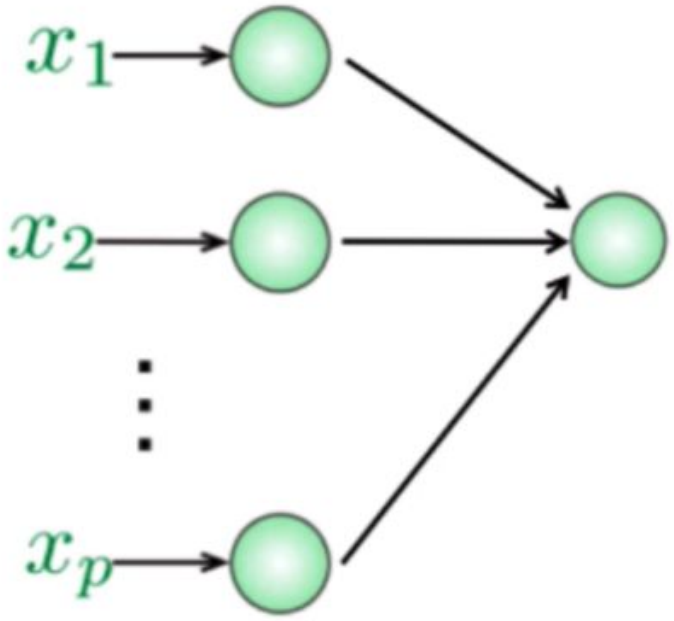
Sensitive to the location of the object in an image!!!!



Disadvantage of FCNN for image classification

No Spatial Information

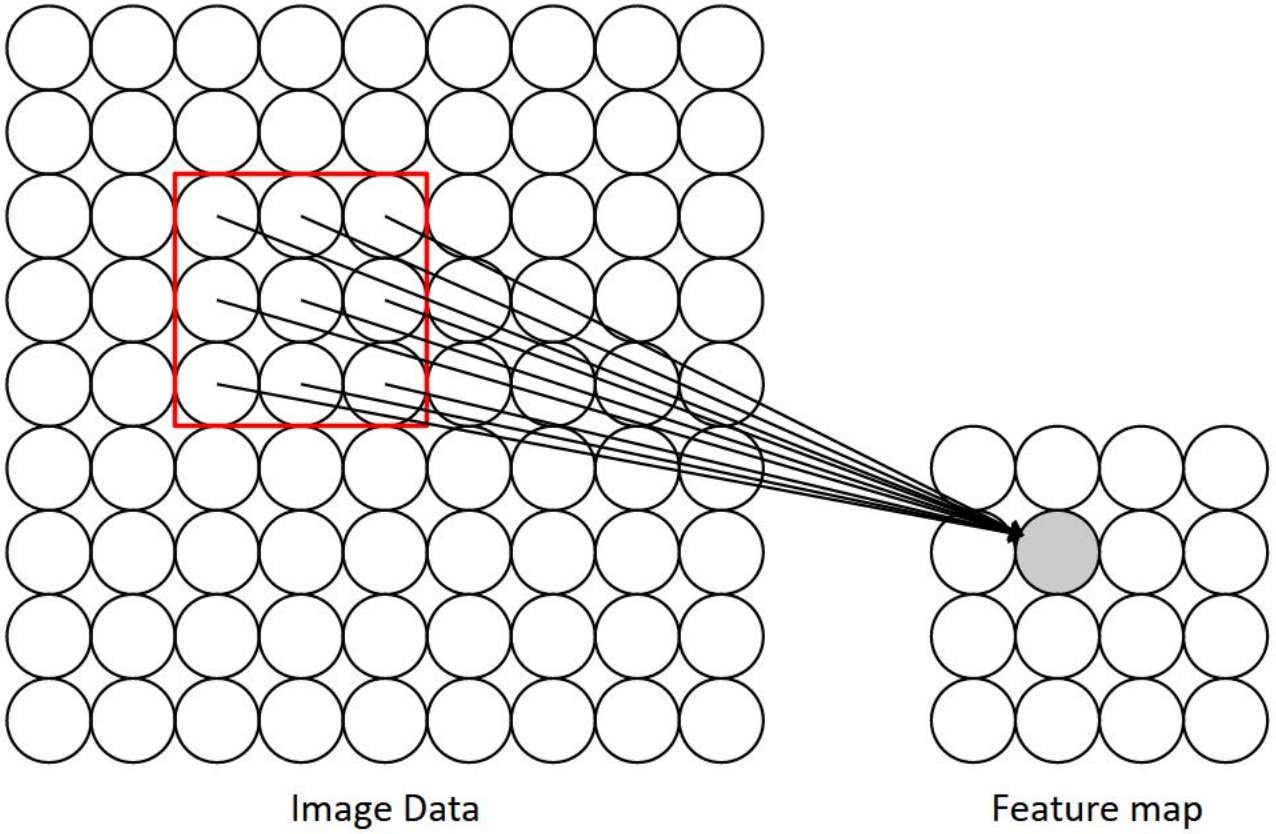
Treats local pixels same as the pixels far apart



Connects all the nodes in the hidden layers to all the nodes in the input layer !

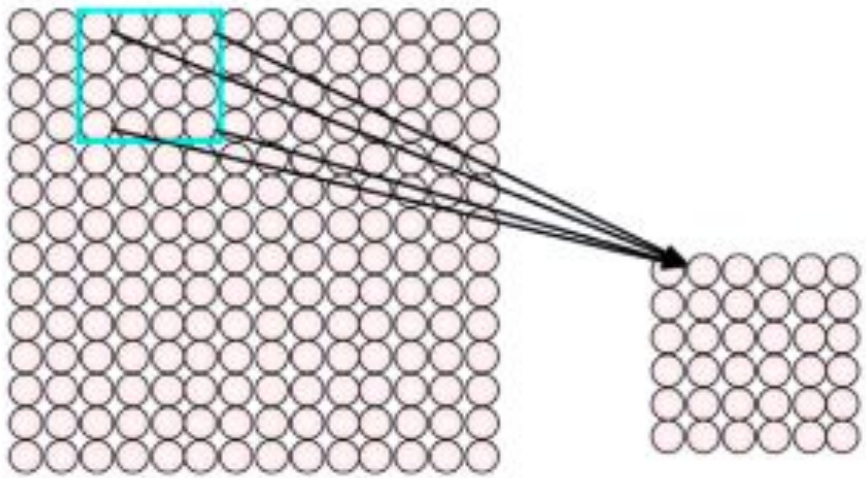
How can we use the spatial features in the input to inform the architecture of the network?

Using Spatial Features (Idea)



- **Idea:** connect patches of input to neurons in hidden layer.
- Neuron connected to region of input. Only "**sees**" these values.

Feature Extraction with Convolution



- Filter of size 4x4 : 16 different weights
- Apply this same filter to 4x4 patches in input
- Shift by 2 pixels for next patch

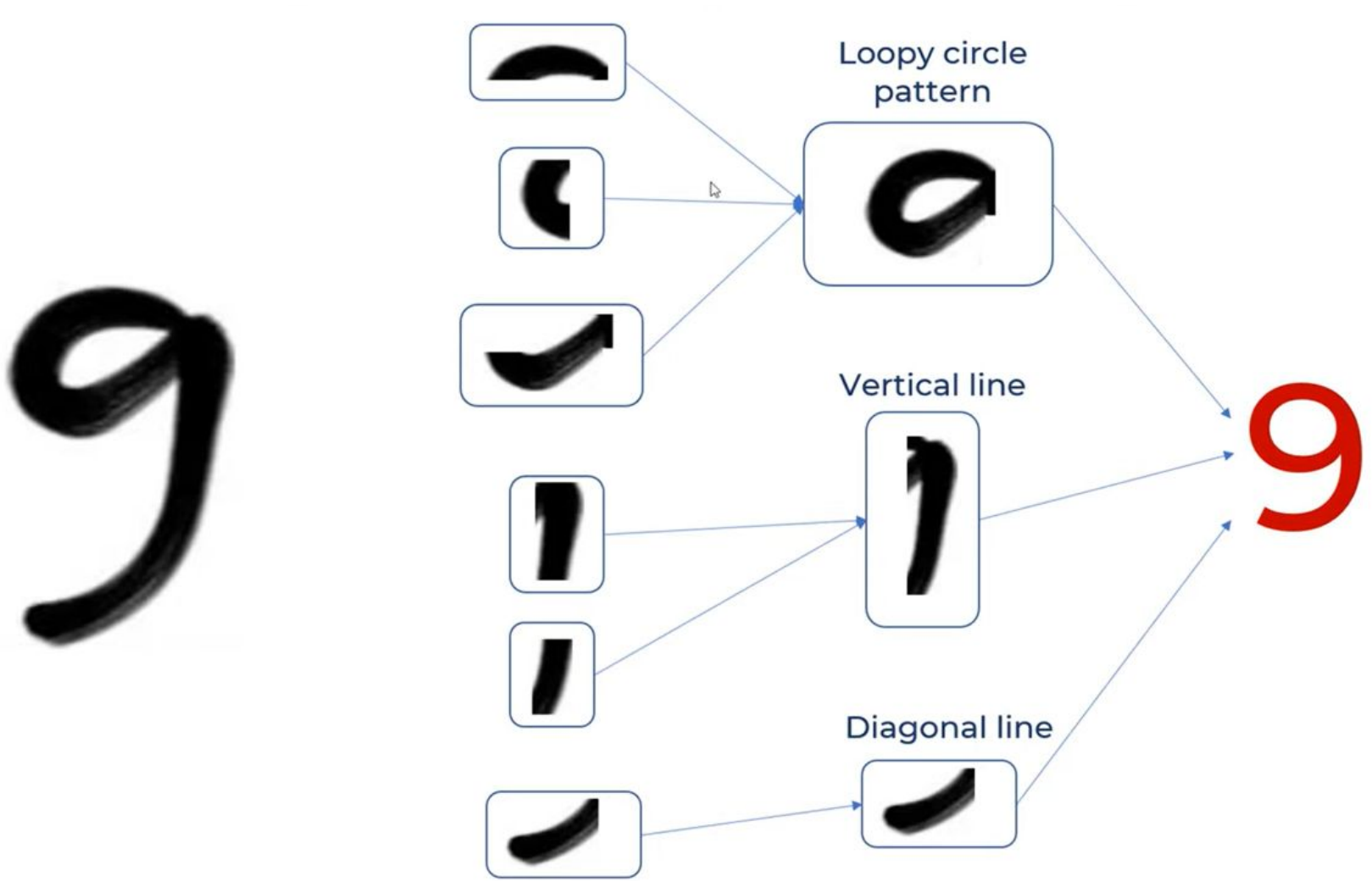
This "patchy" operation is **convolution**

1) Apply a set of weights – a filter – to extract **local features**

2) Use **multiple filters** to extract different features

3) **Spatially share** parameters of each filter

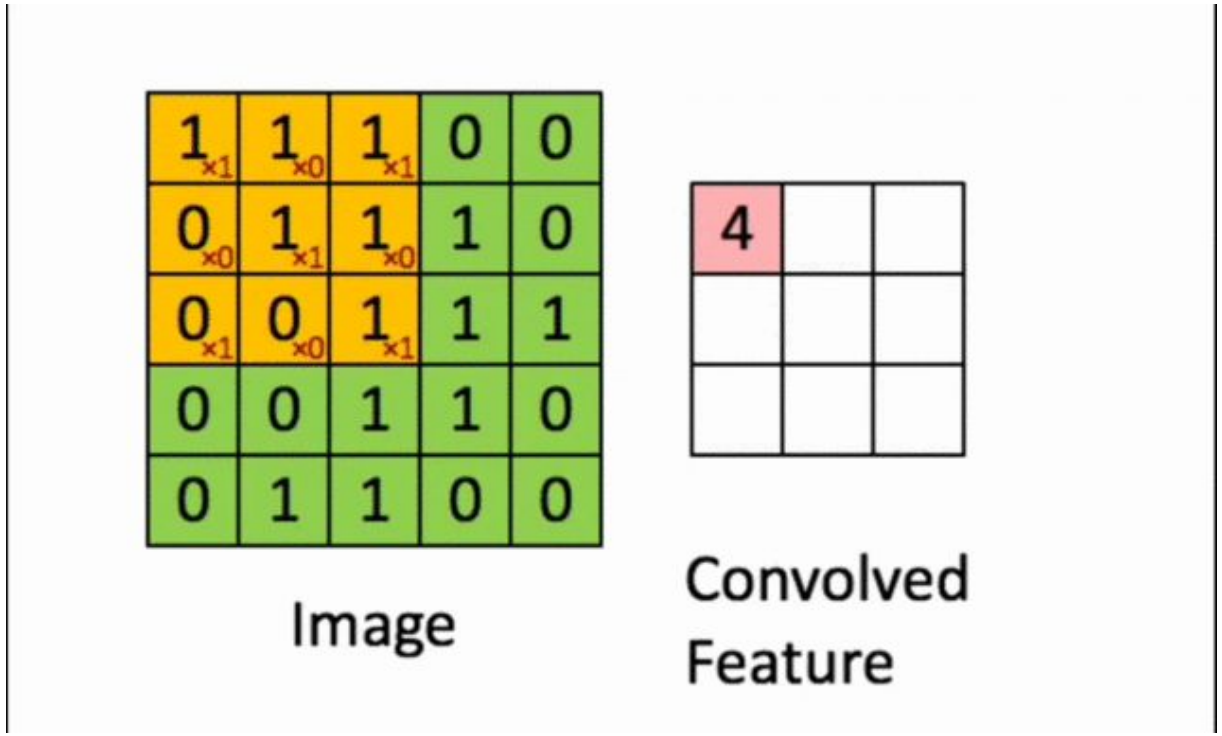
Intuition: Convolution Operation



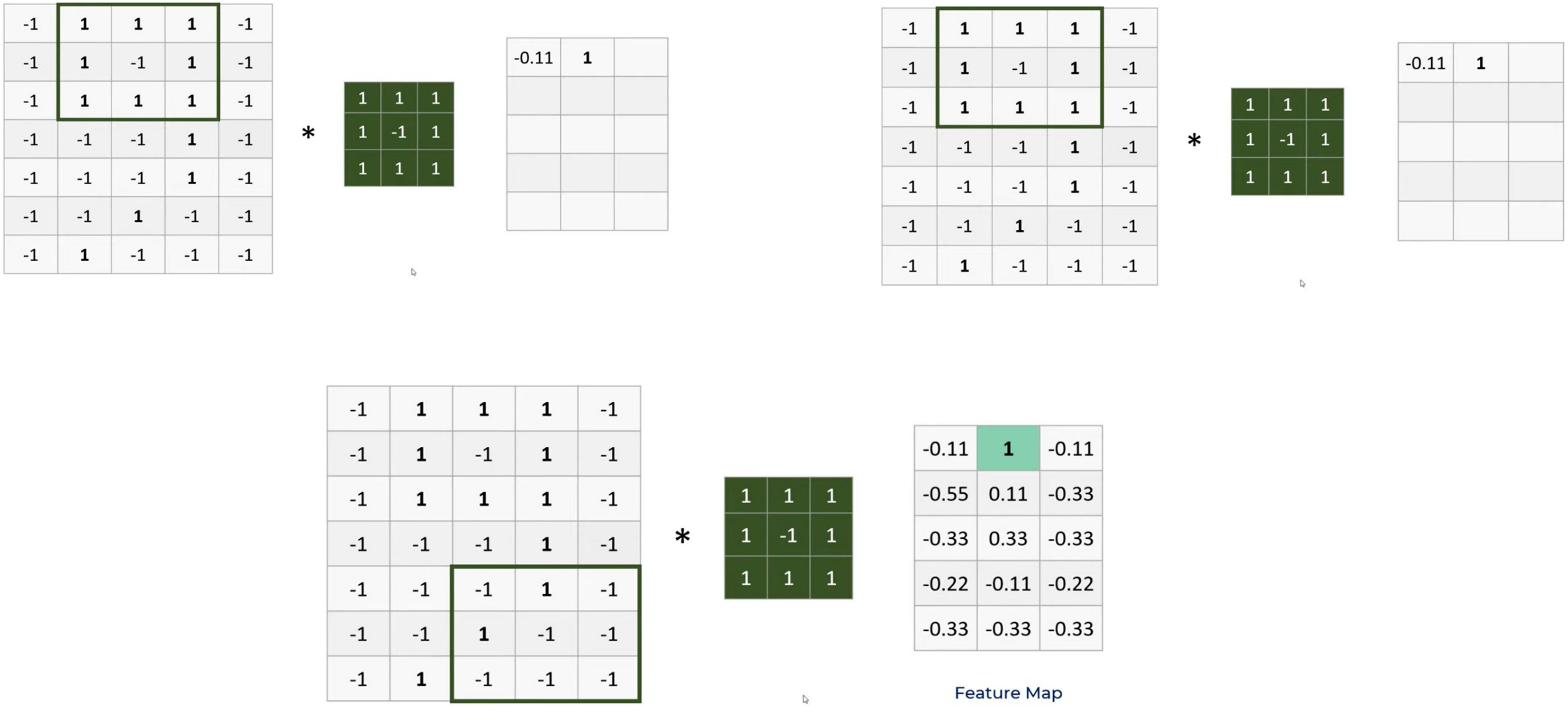
- Detects local patterns (loops, lines, curves)
- Breaks down digit "9" into meaningful features
- Identifies spatial components independently
- Combines detected features for classification
- Preserves spatial relationships between features
- Applies same filters across entire image

Convolution Operation (Mathematically)

We slide the filter over the Input image, element wise multiply and add the output!!



Understanding Convolution Operation



Understanding Convolution Operation

The Loopy Pattern Detector can detect that pattern, all across the input image, no matter the position

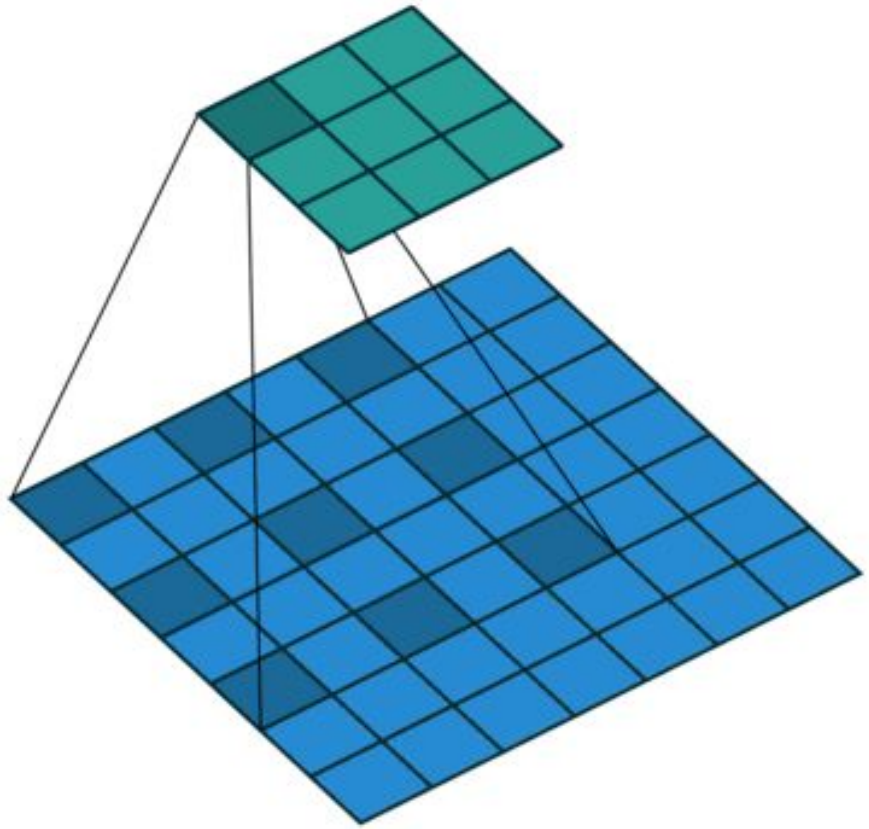
Remember the Terminologies

- **Convolution**: Operation that extracts features using filters

- **Kernel/Filter**: Small matrix that detects specific patterns

- **Feature Map**: Output after applying a filter to input

- **Stride**: Step size when sliding the filter



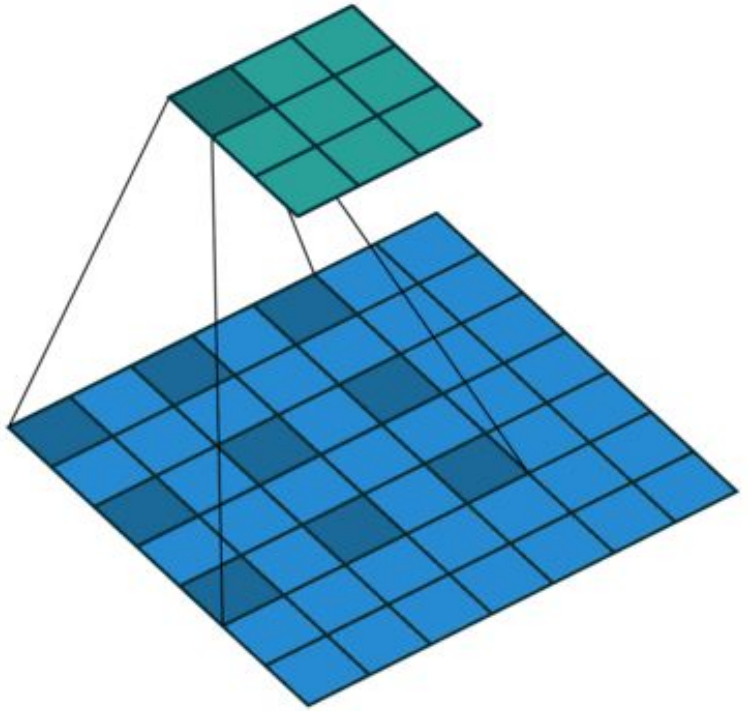
Producing Feature Maps

You can change the weight values in filter to represent different filters which can detect different features in the image

Padding

Preserves pixels at image boundaries during convolution

- Prevents cumulative spatial information loss in deep networks

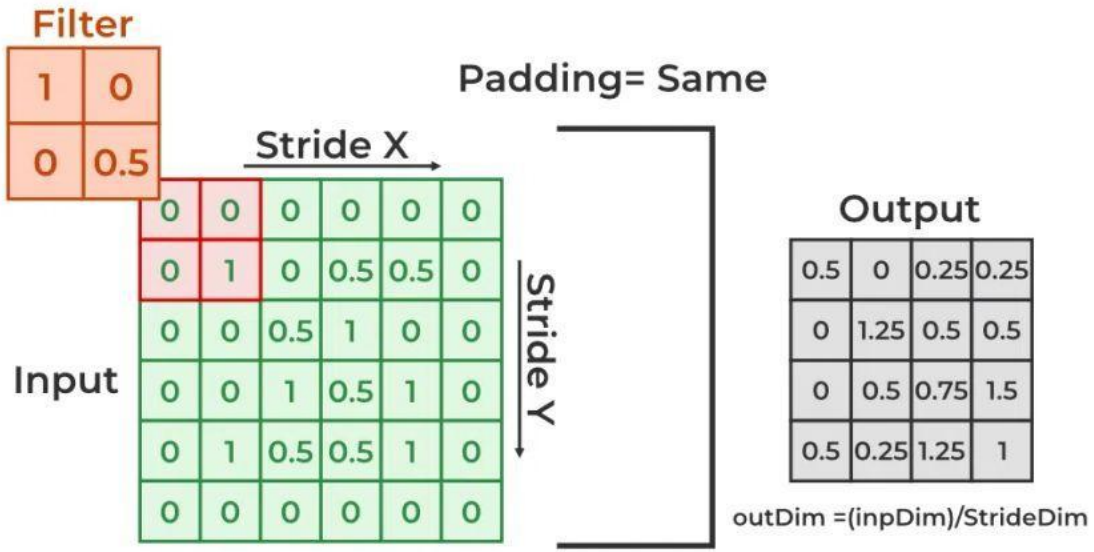


- Types:

Zero Padding: Adds zeros around image borders

Same Padding: Ensures output dimensions match input

Valid Padding: No padding, resulting in smaller output



Output Dimension after Convolution Operation

1. **Filters (K):** Number of feature detectors; controls output depth.
2. **Filter Size (F):** Spatial size (e.g., 3×3, 5×5).
3. **Stride (S):** Step size for filter movement (common: 1).
4. **Padding (P):**
   - **Same Padding (P > 0):** Output size = input size.
   - **Valid Padding (P = 0):** Output shrinks.

Output Shape Calculation:

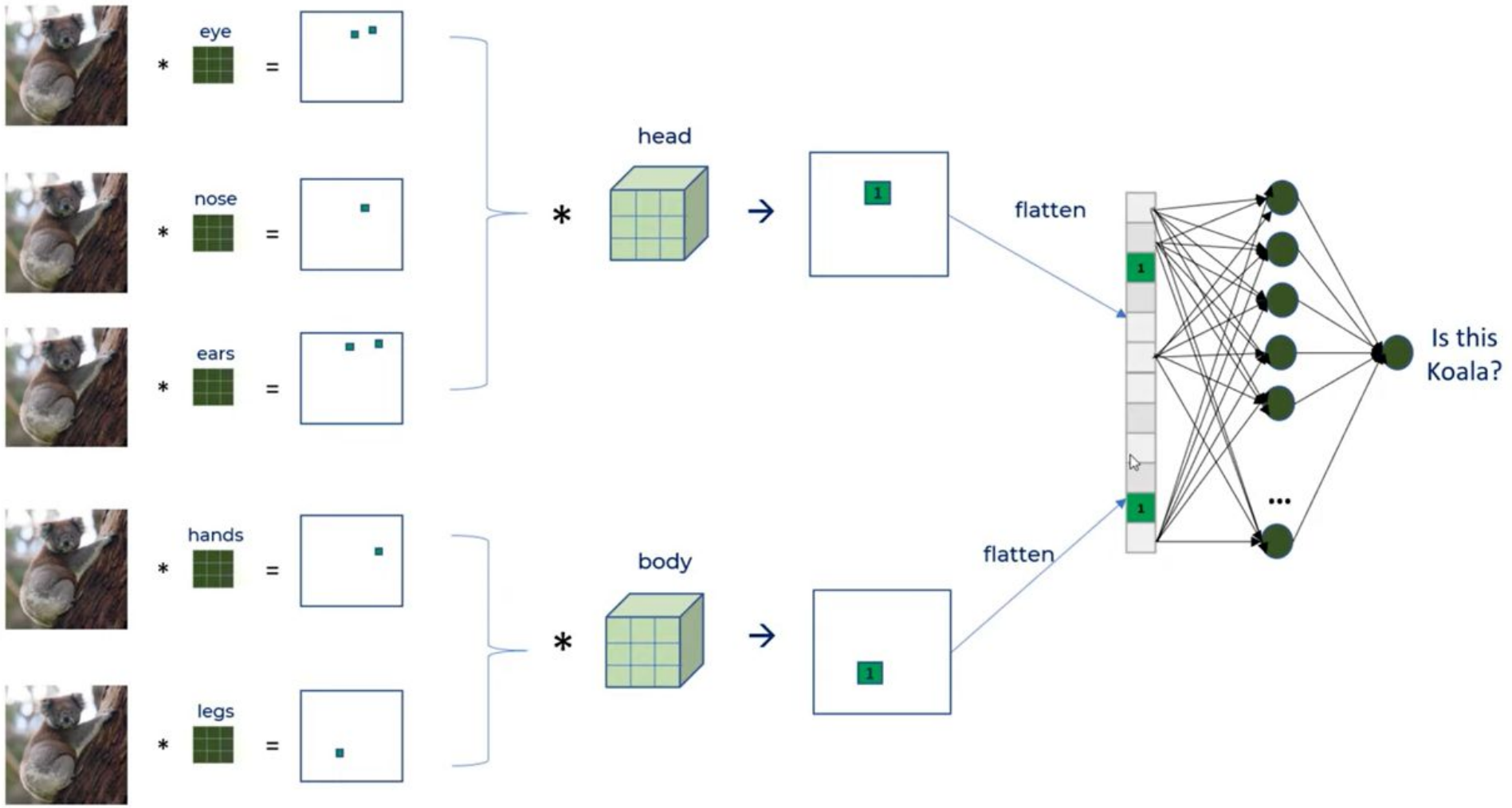
Given an input of size $\mathbf{W_{in} \times H_{in} \times C_{in}}$ the output volume dimensions are:

$$\mathbf{W_{oc} = \frac{(W_{in} - F + 2P)}{S} + 1}$$

$$\mathbf{H_{oc} = \frac{H_{in} - F + 2P}{S} + 1}$$

$\mathbf{C_{oc} = K}$ (Number of filters, which determines the depth of the output)

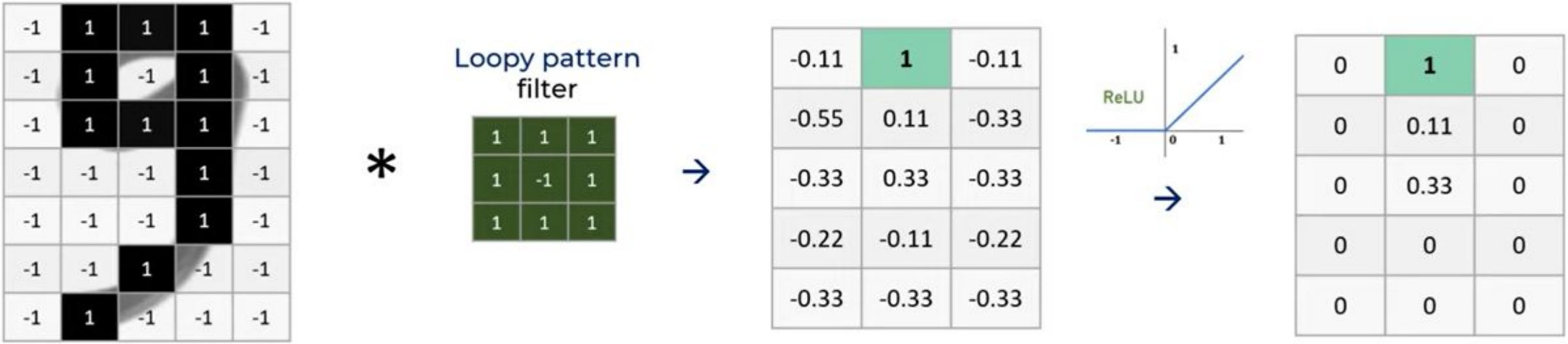
A Bigger Picture (Convolution Layers)



Introducing Non-linearity

- Apply after every convolution operation (i.e., after convolutional layers)
- ReLU: pixel-by-pixel operation that replaces all negative values by zero.

Non-linear operation!



Pooling Layer

- It is a layer inserted **in-between successive Convolution layer** in a CNN.
- It operates similar to convolution operation i.e. by sliding a **filter over feature maps**.
- It down samples the convolved features by **reducing the number of dimensions** of the **feature maps** while **still preserving the most critical feature information**.
- It reduces the amount of parameters and computation in the network and hence also supports in controlling overfitting.
- The most common algorithm used for this operation is called **max pooling**.

Pooling

- Reduced Dimensionality
- Spatial Variance

5	1	3	4
8	2	9	2
1	3	0	1
2	2	2	0

8	**9**
3	2

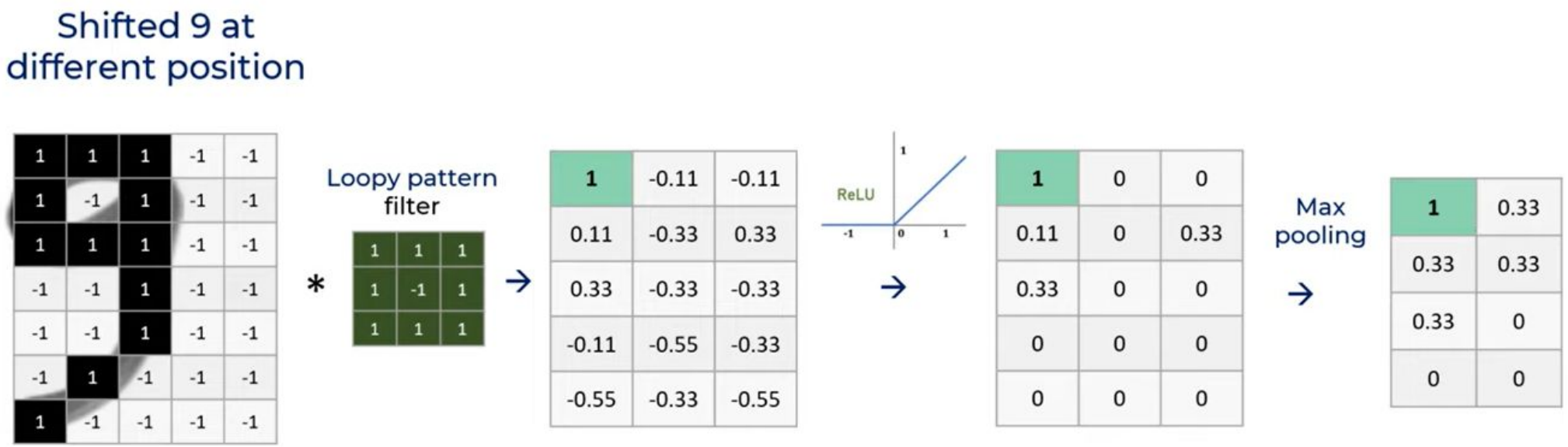
2 by 2 filter with stride = 2

Output Dimensions After Pooling

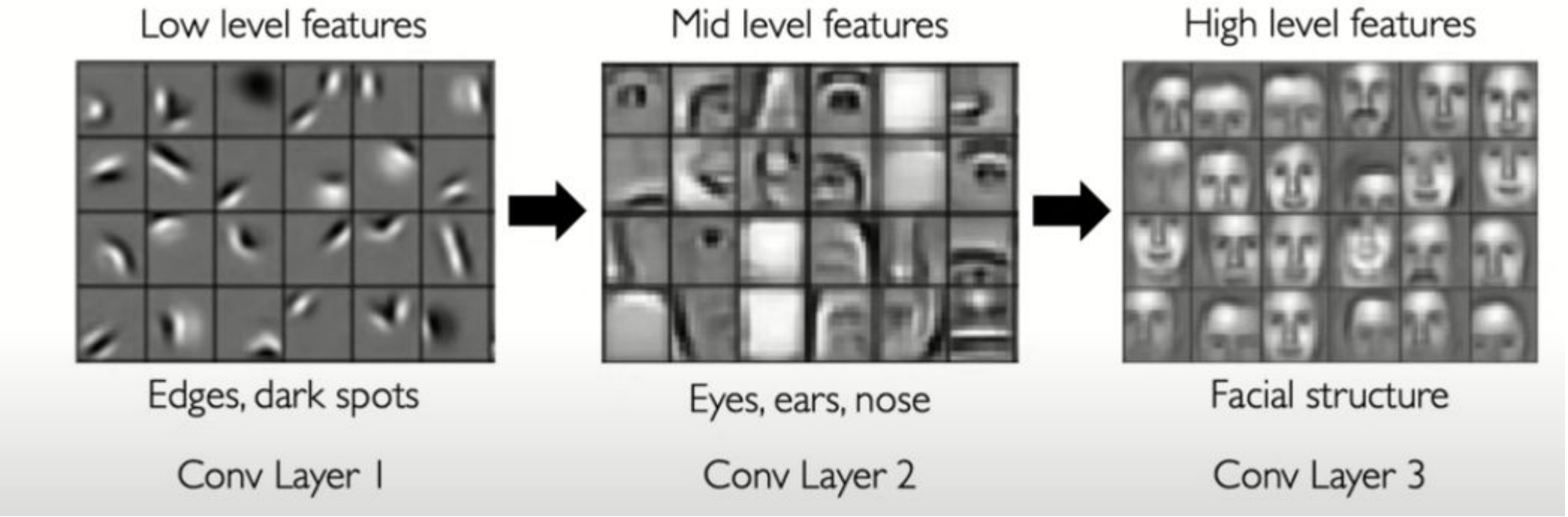
- **Hyperparameters**: Filter Size (F): Typically 2×2
- Stride (S): Usually 2, reduces spatial dimensions by half
- A pooling layer takes an input of volume:
  - $\mathbf{W_{oca} \times H_{oca} \times C_{oca}}$ $\mathbf{\{oca \rightarrow Output\ after\ convolution\ and\ activation.\}}$
- And produces an output of volume:
  - $\mathbf{W_{ocap} \times H_{ocap} \times C_{ocap}}$ $\mathbf{\{ocap \rightarrow Output\ after\ convolution, activation\ and\ pooling.\}}$
- Where:
  - $\mathbf{W_{ocap} = \frac{W_{oca} - F}{S} + 1}$
  - $\mathbf{H_{ocap} = \frac{H_{oca} - F}{S} + 1}$
  - $\mathbf{C_{ocap} = C_{oca}}$ $\mathbf{(depth\ remains\ unchanaged)}$

Story So far!

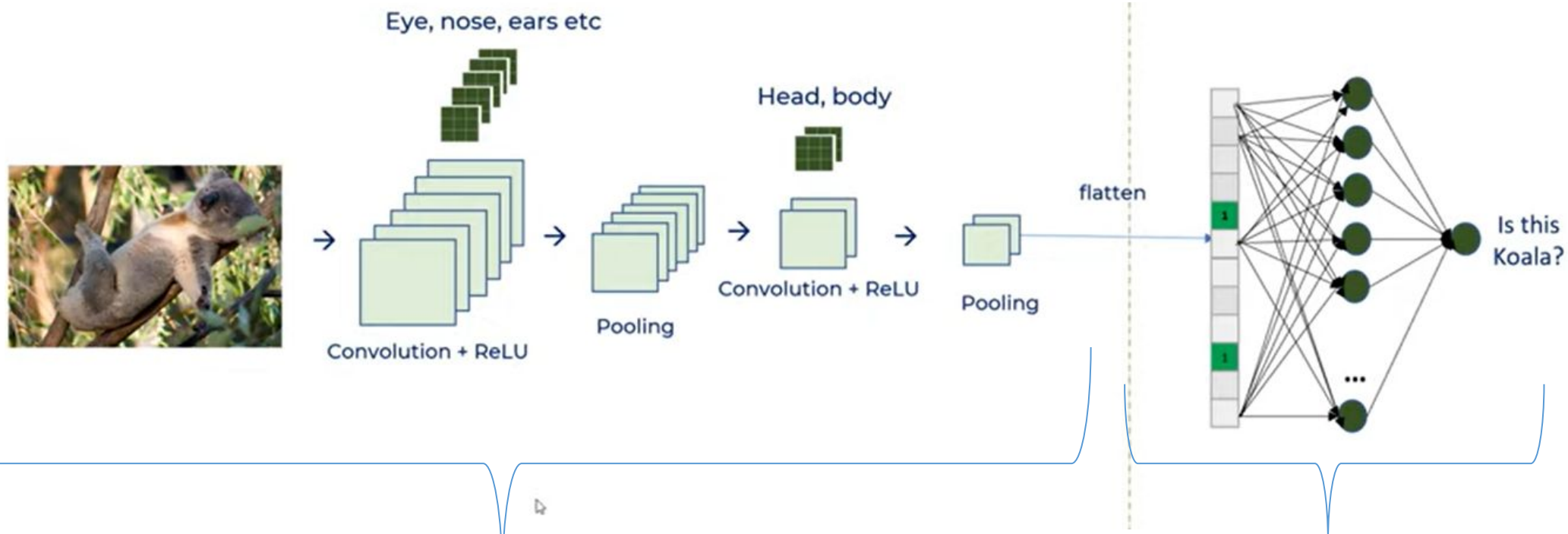
Convolution Layer
ReLU Activation
Pooling



Representation Learning in Deep CNN

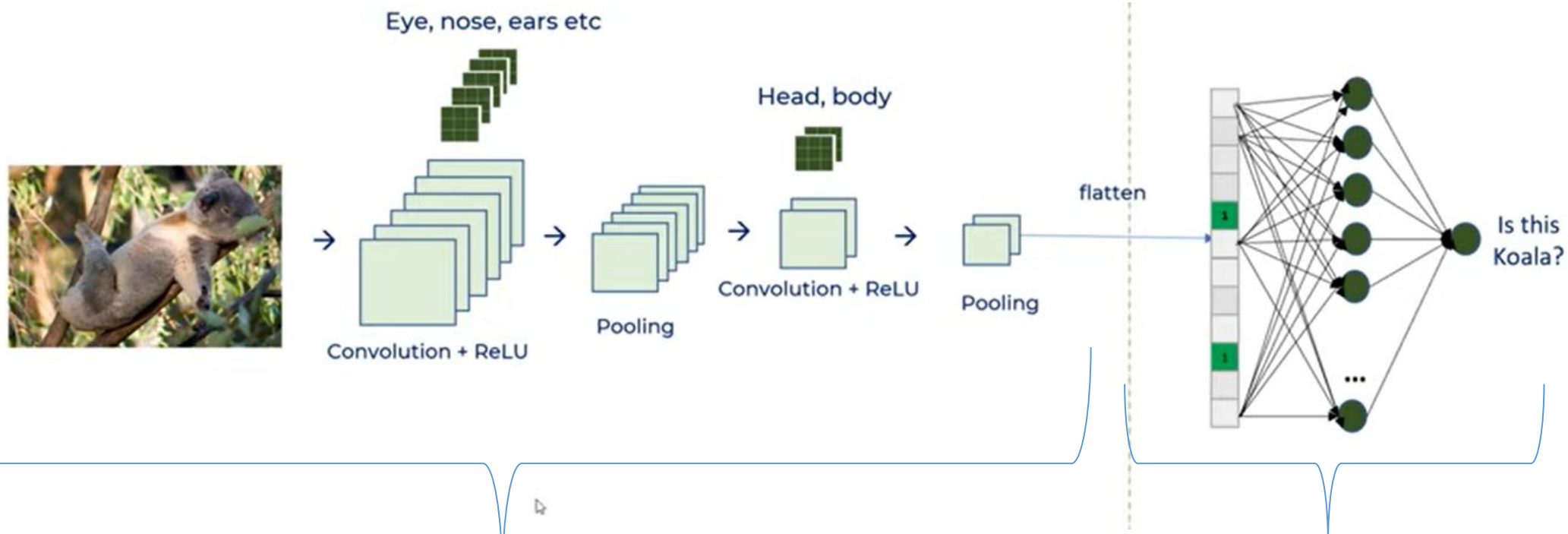


CNNS For Classification: Feature Learning



- Learn features in input image through **convolution**
- Introduce **non-linearity** through activation function (real-world data is non-linear!)
- Reduce dimensionality and preserve spatial invariance with **pooling**

CNNS For Classification: Classification

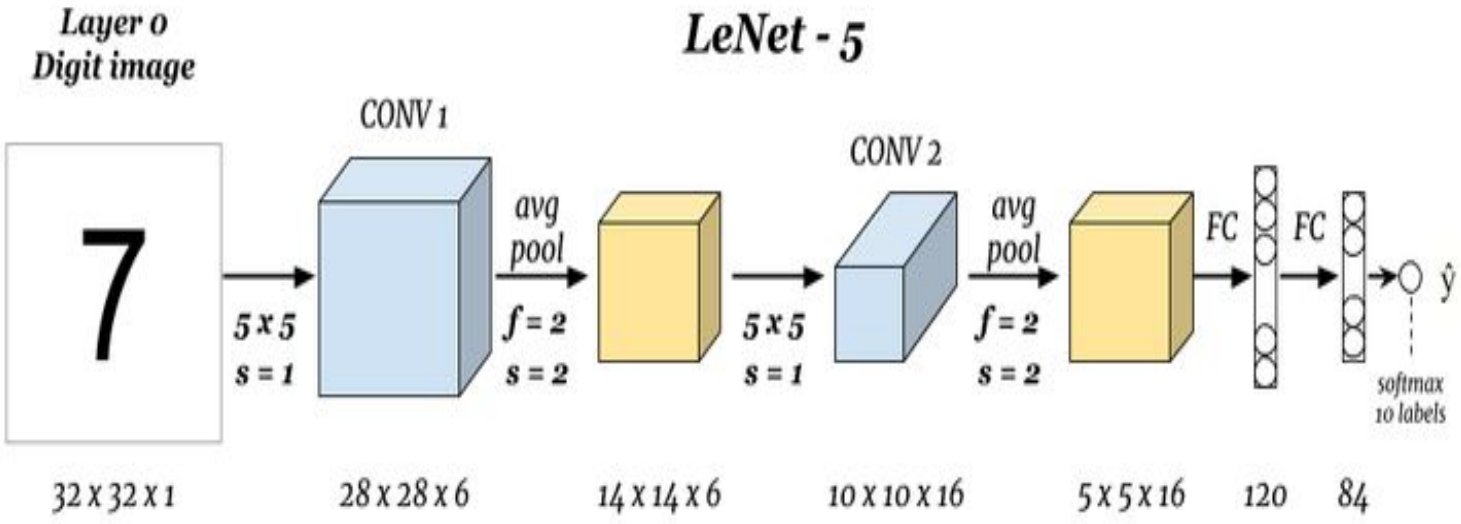


- CONV and POOL layers output high-level features of input
- Fully connected layer uses these features for classifying input image
- Express output as probability of image belonging to a particular class

Case Studies-CNN Architecture.

Classic Architecture-LeNet-5:
[Conv, ReLU, Pool]*N,□ flatten,[FC, ReLU]*N, FC □ Softmax.

Layer	Output Size	No Parameters
Flatten	5*5*16 (120)	6*6*5*5
Linear(120□80)	80	120*80.



This is the architecture of LeNet-5 created by **Yann LeCun** in **1998** and was widely used for hand written digits recognition (MNIST).


4627
4 6 2 7

Training The CNN

Forward Computation @ Convolutional Layer.

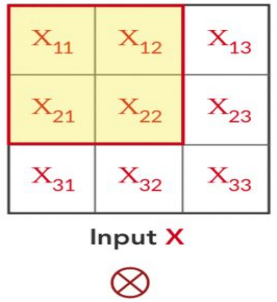
$$\begin{bmatrix} O_{11} & O_{12} \\ O_{21} & O_{22} \end{bmatrix} = \text{Convolution}\left(\begin{bmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & X_{22} & X_{23} \\ X_{31} & X_{32} & X_{33} \end{bmatrix}, \begin{bmatrix} F_{11} & F_{12} \\ F_{21} & F_{22} \end{bmatrix} \right)$$

$$O_{11} = F_{11}X_{11} + F_{12}X_{12} + F_{21}X_{21} + F_{22}X_{22}$$

$$O_{12} = F_{11}X_{12} + F_{12}X_{13} + F_{21}X_{22} + F_{22}X_{23}$$

$$O_{21} = F_{11}X_{21} + F_{12}X_{22} + F_{21}X_{31} + F_{22}X_{32}$$

$$O_{22} = F_{11}X_{22} + F_{12}X_{23} + F_{21}X_{32} + F_{22}X_{33}$$



$X_{11}F_{11}$	$X_{12}F_{12}$	X_{13}
$X_{21}F_{21}$	$X_{22}F_{22}$	X_{23}
X_{31}	X_{32}	X_{33}

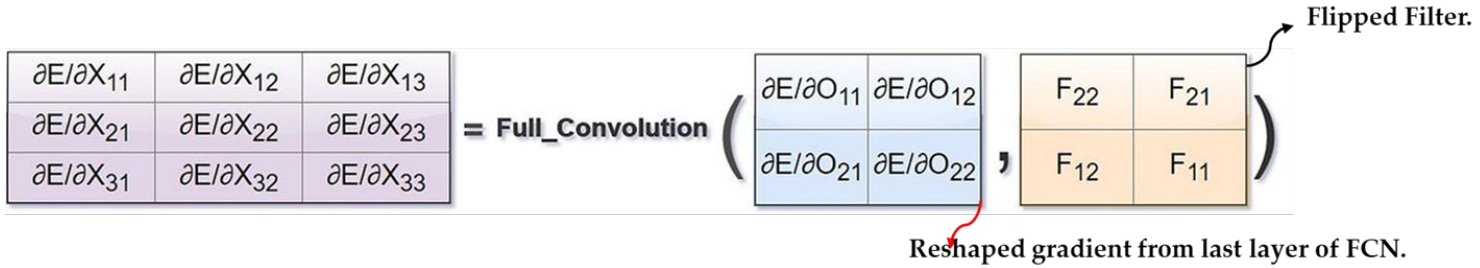
$$O_{11} = X_{11}F_{11} + X_{12}F_{12} + X_{21}F_{21} + X_{22}F_{22}$$

Image from Internet: Subject to copyright.

Same Operation happens at every Layer, the inputs shall change depending on the layer.

Backward Pass @ Convolution Layer.

- Computing the gradient w.r.t the filter F:
  - This tells us how much each filter contributes to the loss:
  - Calculated as a convolution between input X and the gradient of the error $\delta = \frac{\partial E}{\partial O}$.



$$\frac{\partial E}{\partial X_{11}} = \frac{\partial E}{\partial O_{11}} F_{11} + \frac{\partial E}{\partial O_{12}} 0 + \frac{\partial E}{\partial O_{21}} 0 + \frac{\partial E}{\partial O_{22}} 0$$
$$\frac{\partial E}{\partial X_{12}} = \frac{\partial E}{\partial O_{11}} F_{12} + \frac{\partial E}{\partial O_{12}} F_{11} + \frac{\partial E}{\partial O_{21}} 0 + \frac{\partial E}{\partial O_{22}} 0$$
$$\frac{\partial E}{\partial X_{13}} = \frac{\partial E}{\partial O_{11}} 0 + \frac{\partial E}{\partial O_{12}} F_{12} + \frac{\partial E}{\partial O_{21}} 0 + \frac{\partial E}{\partial O_{22}} 0$$
$$\frac{\partial E}{\partial X_{21}} = \frac{\partial E}{\partial O_{11}} F_{21} + \frac{\partial E}{\partial O_{12}} 0 + \frac{\partial E}{\partial O_{21}} F_{11} + \frac{\partial E}{\partial O_{22}} 0$$
$$\frac{\partial E}{\partial X_{22}} = \frac{\partial E}{\partial O_{11}} F_{22} + \frac{\partial E}{\partial O_{12}} F_{21} + \frac{\partial E}{\partial O_{21}} f_{12} + \frac{\partial E}{\partial O_{22}} F_{11}$$
$$\frac{\partial E}{\partial X_{23}} = \frac{\partial E}{\partial O_{11}} 0 + \frac{\partial E}{\partial O_{12}} F_{22} + \frac{\partial E}{\partial O_{21}} 0 + \frac{\partial E}{\partial O_{22}} F_{11}$$
$$\frac{\partial E}{\partial X_{31}} = \frac{\partial E}{\partial O_{11}} 0 + \frac{\partial E}{\partial O_{12}} 0 + \frac{\partial E}{\partial O_{21}} F_{21} + \frac{\partial E}{\partial O_{22}} 0$$
$$\frac{\partial E}{\partial X_{32}} = \frac{\partial E}{\partial O_{11}} 0 + \frac{\partial E}{\partial O_{12}} 0 + \frac{\partial E}{\partial O_{21}} F_{22} + \frac{\partial E}{\partial O_{22}} F_{21}$$

Backward Pass @ Convolution Layer.

- Computing the gradient w.r.t the input X:
  - This tells us how much each pixel in the input contributes to the loss:
  - Calculated as a convolution between flipped filter F and the gradient of the error $\delta = \frac{\partial E}{\partial O}$.

$$\begin{bmatrix} \partial E/\partial F_{11} & \partial E/\partial F_{12} \\ \partial E/\partial F_{21} & \partial E/\partial F_{22} \end{bmatrix} = \text{Convolution}\left(\begin{bmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & X_{22} & X_{23} \\ X_{31} & X_{32} & X_{33} \end{bmatrix}, \begin{bmatrix} \partial E/\partial O_{11} & \partial E/\partial O_{12} \\ \partial E/\partial O_{21} & \partial E/\partial O_{22} \end{bmatrix} \right)$$

This is the reshaped gradient output from FCN

$$\frac{\partial E}{\partial F_{11}} = \frac{\partial E}{\partial O_{11}} X_{11} + \frac{\partial E}{\partial O_{12}} X_{12} + \frac{\partial E}{\partial O_{21}} X_{21} + \frac{\partial E}{\partial O_{22}} X_{22}$$

$$\frac{\partial E}{\partial F_{12}} = \frac{\partial E}{\partial O_{11}} X_{12} + \frac{\partial E}{\partial O_{12}} X_{13} + \frac{\partial E}{\partial O_{21}} X_{22} + \frac{\partial E}{\partial O_{22}} X_{23}$$

$$\frac{\partial E}{\partial F_{21}} = \frac{\partial E}{\partial O_{11}} X_{21} + \frac{\partial E}{\partial O_{12}} X_{22} + \frac{\partial E}{\partial O_{21}} X_{31} + \frac{\partial E}{\partial O_{22}} X_{32}$$

$$\frac{\partial E}{\partial F_{22}} = \frac{\partial E}{\partial O_{11}} X_{22} + \frac{\partial E}{\partial O_{12}} X_{23} + \frac{\partial E}{\partial O_{21}} X_{32} + \frac{\partial E}{\partial O_{22}} X_{33}$$

References

- https://shwetarkadam25.medium.com/cnn-series-part-2-what-is-meant-by-convolution-c504f38c42b
- cnn explainer
- Convolutional Network Demo from 1989